INFO20003 Database Systems

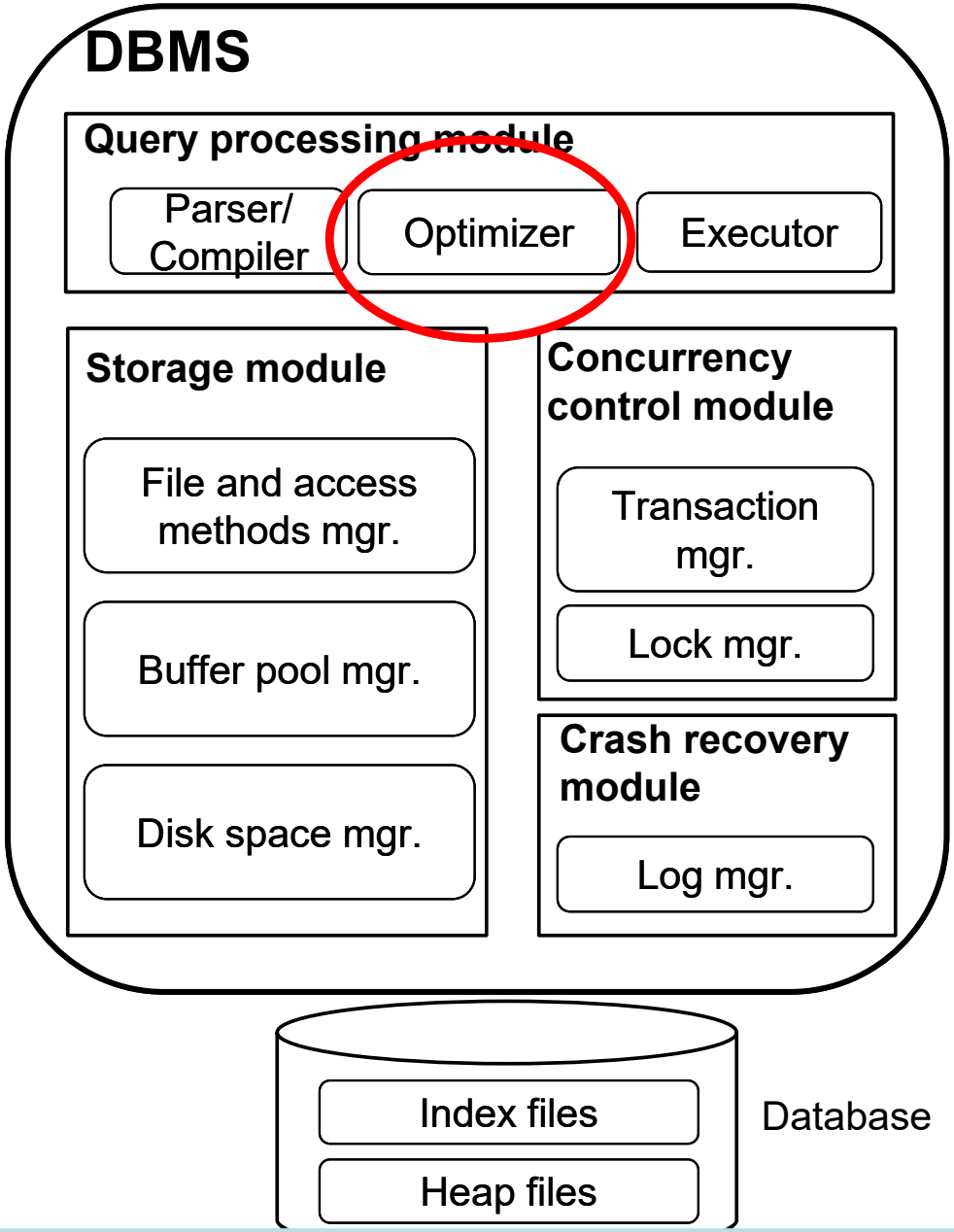
Sem 2, 2023

Lecturer: Egemen Tanin, Prof.
School of Computing and Information Systems

Lecture 13
Query Optimization Part I

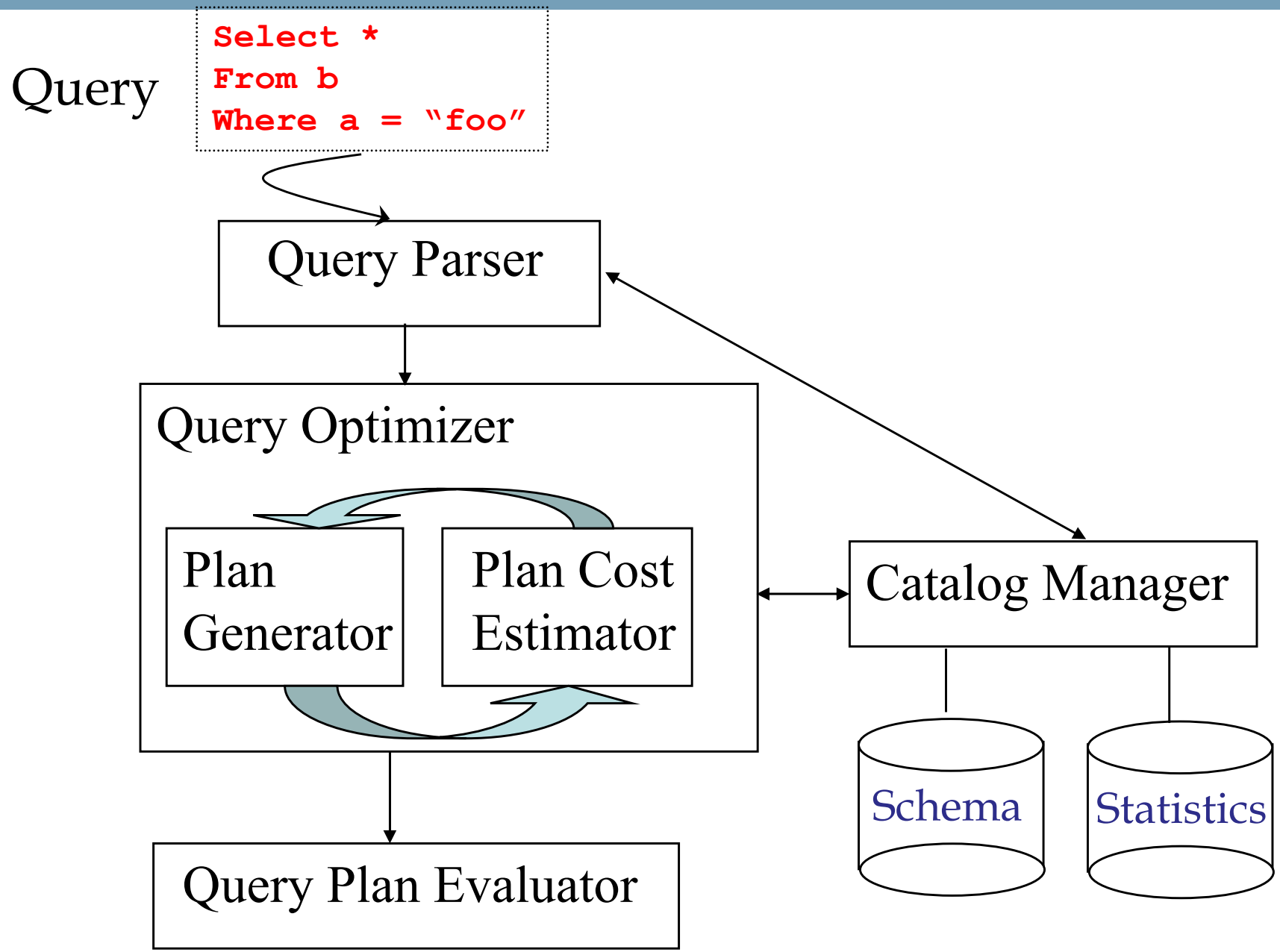
Week 7

Components of a DBMS



Readings: Chapter 12 and 15, Ramakrishnan & Gehrke, Database Systems

Query Processing Workflow



© University of Melbourne

Why do Query Optimization?

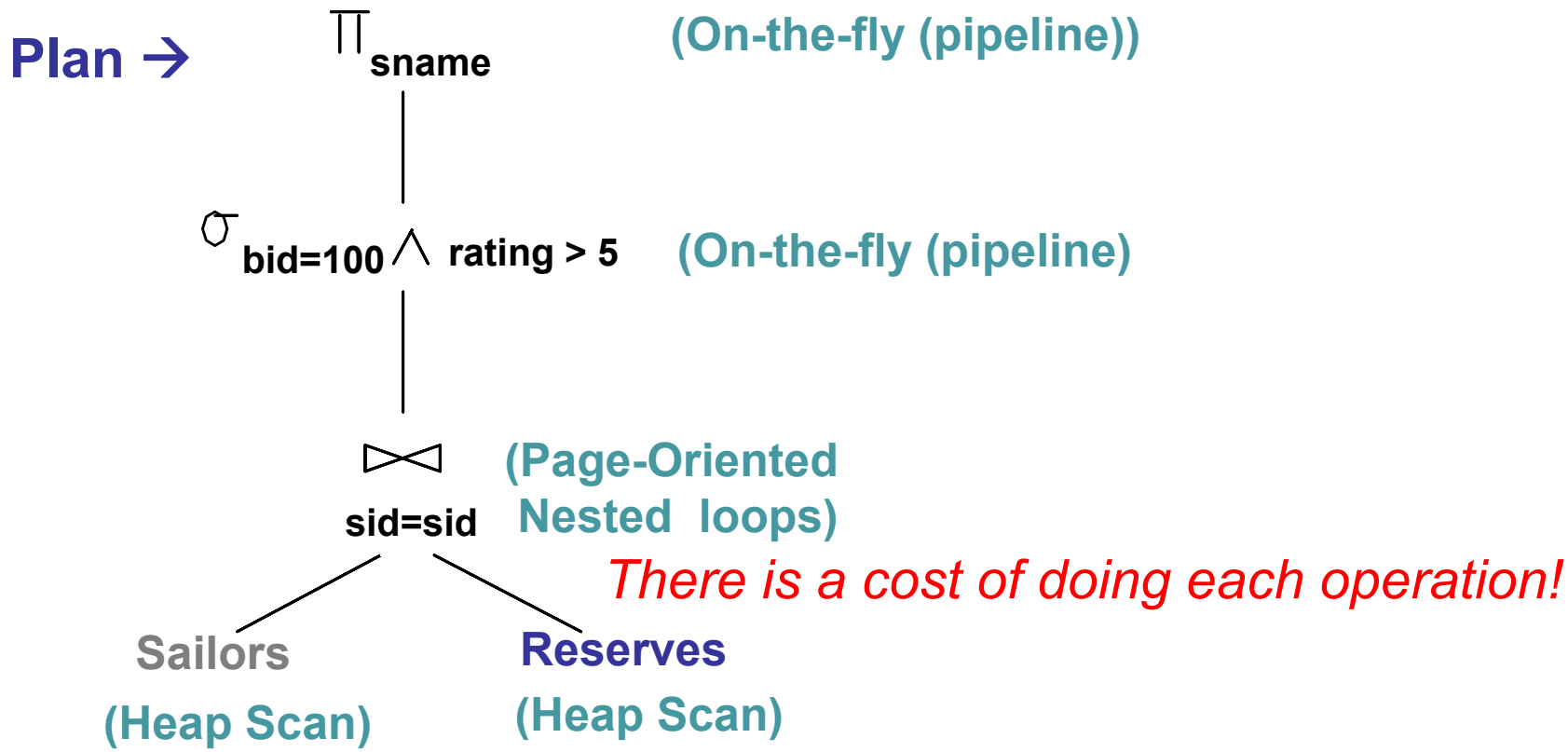
- Typically there are **many ways of executing a given query**, all **giving the same answer**
- Cost **varies enormously** for alternative methods
- Query optimization aims to **find the execution strategy with the lowest cost**

© University of Melbourne

Query Plan is at the core

- Is a tree, with relational algebra operators as nodes and access paths as leaves
- Each operator labeled with a choice of an algorithm

SELECT sname from Sailors NATURAL JOIN Reserves
WHERE bid = 100 and rating > 5



© University of Melbourne

A Familiar Schema for Our Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: dates, *rname*: string)

Boats (*bid*: integer, *bname*: string, *color*: string)

© University of Melbourne

Query Optimization Steps

Example:

```
SELECT  S.sname
  FROM  Reserves R, Sailors S
WHERE  R.sid=S.sid AND
   R.bid=100 AND S.rating>5
```

Query optimization steps:

1. Query first broken into “blocks”
2. Each block converted to relational algebra
3. Then, for each block, several alternative query plans are considered
4. Plan with the lowest estimated cost is selected

© University of Melbourne

Step 1: Break query into query blocks

- Query block is any statement starting with select
- Query block = unit of optimization
- Typically inner most block is optimized first, then moving towards outers

Outer block

```
SELECT  S.sname
FROM  Sailors S
WHERE  S.age IN
    (SELECT  MAX (S2.age)
      FROM  Sailors S2
      GROUP BY  S2.rating)
```

Nested block

© University of Melbourne

Step 2: Convert query block into relational algebra expression

Query:

```
SELECT  S.sid
FROM  Sailors S, Reserves R, Boats B
WHERE  S.sid = R.sid AND R.bid = B.bid AND B.color = "red"
```

Relational algebra:

$$\pi_{\text{S.sid}}(\sigma_{\text{B.color} = \text{“red”}}(\text{Sailors} \bowtie \text{Reserves} \bowtie \text{Boats}))$$

© University of Melbourne

Step 3: Relational Algebra Equivalences

- *Selections*: $\sigma_{c_1 \wedge \cdots \wedge c_n}(R) \equiv \sigma_{c_1}\left(\ldots\left(\sigma_{c_n}(R)\right)\right)$ *(Cascade)*

  $\sigma_{c_1}\left(\sigma_{c_2}(R)\right) \equiv \sigma_{c_2}\left(\sigma_{c_1}(R)\right)$ *(Commute)*

- *Projections:* $\pi_{a_1}(R) \equiv \pi_{a_1}\left(\ldots\left(\pi_{a_n}(R)\right)\right)$ *(Cascade)*

  a_i is a set of attributes of R and $a_i \subseteq a_{i+1}$ for $i = 1 \ldots n-1$

- These equivalences allow us to 'push' selections and projections ahead of joins.

© University of Melbourne

Examples

Selection:

$$\sigma_{age<18 \wedge rating>5}(\text{Sailors})$$

$$\leftrightarrow \sigma_{age<18}(\sigma_{rating>5}(\text{Sailors}))$$

$$\leftrightarrow \sigma_{rating>5}(\sigma_{age<18}(\text{Sailors}))$$

Projection:

$$\cancel{\pi_{age,rating}(\text{Sailors}) \leftrightarrow \pi_{age}(\pi_{rating}(\text{Sailors}))}$$

$$\pi_{age,rating}(\text{Sailors}) \leftrightarrow \pi_{age,rating}(\pi_{age,rating,sid}(\text{Sailors}))$$

© University of Melbourne

Another Equivalence

- A projection commutes with a selection that only uses attributes retained by the projection

$$\pi_{age,\ rating,\ sid}\ (\sigma_{age<18\ \wedge\ rating>5}\ (Sailors))$$

$$\leftrightarrow\ \sigma_{age<18\ \wedge\ rating>5}\ (\pi_{age,\ rating,\ sid}\ (Sailors))$$

$$\pi_{age,\ sid}\ (\sigma_{age<18\ \wedge\ rating>5}\ (Sailors))$$

$$\cancel{\leftrightarrow\ \sigma_{age<18\ \wedge\ rating>5}\ (\pi_{age,\ sid}\ (Sailors))}$$

© University of Melbourne

Equivalences Involving Joins

$$R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T \qquad \textit{(Associative)}$$

$$(R \bowtie S) \equiv (S \bowtie R) \qquad \textit{(Commutative)}$$

- These equivalences allow us to choose **different join orders**

© University of Melbourne

Mixing Joins with Selections & Projections

- Converting selection + cross-product to join

$$\sigma_{S.sid = R.sid} (\text{Sailors} \times \text{Reserves})$$

$$\leftrightarrow \text{Sailors} \bowtie_{S.sid = R.sid} \text{Reserves}$$

- Selection on just attributes of S commutes with R $\bowtie$ S

$$\sigma_{S.age<18} (\text{Sailors} \bowtie_{S.sid = R.sid} \text{Reserves})$$

$$\leftrightarrow (\sigma_{S.age<18} (\text{Sailors})) \bowtie_{S.sid = R.sid} \text{Reserves}$$

- We can also "push down" projection (*but be careful*…)

$$\pi_{S.sname} (\text{Sailors} \bowtie_{S.sid = R.sid} \text{Reserves})$$

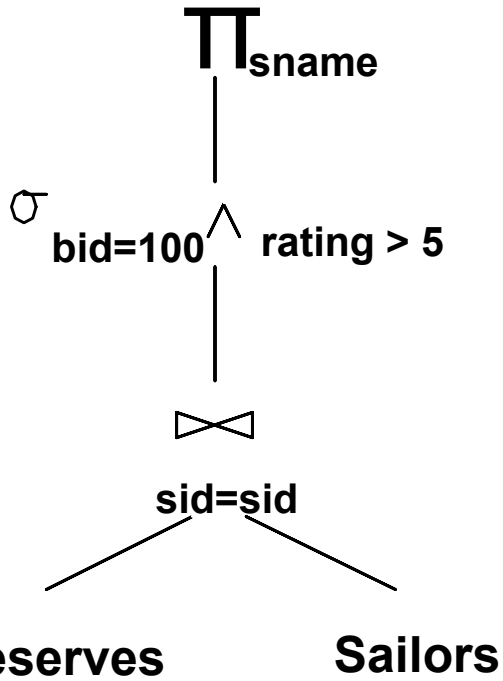
$$\leftrightarrow \pi_{S.sname} (\pi_{sname,sid}(\text{Sailors}) \bowtie_{S.sid = R.sid} \pi_{sid}(\text{Reserves}))$$

© University of Melbourne

Recall: Query Optimization Steps

1. Query first broken into "blocks"
2. Each block converted to relational algebra
3. Then, for each block, several alternative query plans are considered
4. Plan with lowest estimated cost is selected

```
SELECT  S.sname
FROM  Reserves R, Sailors S
WHERE  R.sid=S.sid AND
   R.bid=100 AND S.rating>5
```



$$\pi_{(sname)}\sigma_{(bid=100 \wedge rating > 5)} (Reserves \bowtie Sailors)$$

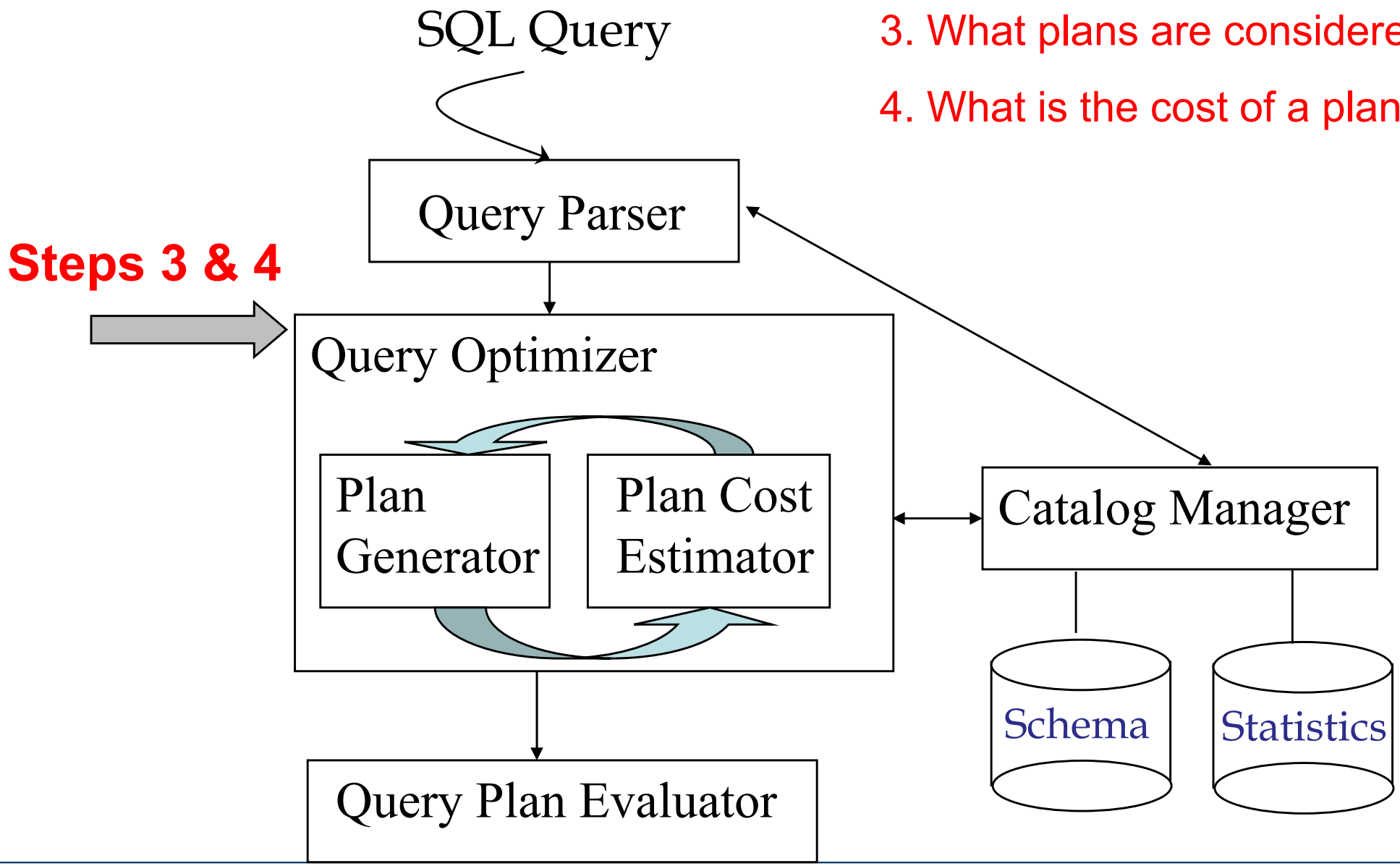
Can we do better here?

© University of Melbourne

Cost-based Query Optimization

3. What plans are considered?

4. What is the cost of a plan?



© University of Melbourne

Cost Estimation

- **Estimate cost for each plan considered**:
  - Must estimate *cost* of each operation in plan tree
    - Depends on input cardinalities
  - Must estimate *size of result* for each operation in tree
    - Use information about input relations (from the system catalogs), and apply certain well established rules

© University of Melbourne

Statistics and Catalogs

- To decide on the cost, the optimizer needs information about the relations and indexes involved. This information is stored in the system **catalogs**

- **Catalogs** typically contain at least:
  - # tuples (**NTuples**) and # pages (**NPages**) per relation
  - # distinct key values (**NKeys**) for each index (or relation attribute)
  - low/high key values (**Low/High**) for each index (or relation attribute)
  - Index height (**Height(I)**) for each tree index
  - # index pages (**NPages(I)**) for each index

- Statistics in catalogs are updated periodically

© University of Melbourne

Result size estimation

- Consider a query block:

```
SELECT  attribute list
  FROM  relation list
WHERE  predicate1 AND ... AND predicate_k
```

- **Maximum number of tuples in the result is the product of the cardinalities of relations** in the **FROM** clause

- **Reduction factor (RF)** associated with each predicate reflects the impact of the predicate in reducing the result size. RF is also called selectivity.

© University of Melbourne

Result size estimation calculations

- Single table selection:

$$\textbf{ResultSize} = \mathrm{N}\boldsymbol{Tuples}(\boldsymbol{R}) \prod_{\boldsymbol{i=1..n}} \boldsymbol{RF_i}$$

- Joins (over k tables):

$$\textbf{ResultSize} = \prod_{\boldsymbol{j=1..k}} \boldsymbol{NTuples}(\boldsymbol{R_j}) \prod_{\boldsymbol{i=1..n}} \boldsymbol{RF_i}$$

- If there are no selections (no predicates), reduction factors are simply ignored, i.e. they are equal to 1

© University of Melbourne

Calculating Reduction Factors (RF)

- Depend on the type of the predicate:

1. Col = value
   RF = 1/NKeys(Col)

2. Col > value
   RF = (High(Col) – value) / (High(Col) – Low(Col))

3. Col < value
   RF = (val – Low(Col)) / (High(Col) – Low(Col))

4. Col_A = Col_B (for joins)
   RF = 1/ (Max (NKeys(Col_A), NKeys(Col_B)))

5. In no information about Nkeys or interval, use a “magic number” 1/10
   RF = 1/10

© University of Melbourne

Let's try to calculate…

Sailors (S): NTuples(S) =1000, Nkeys(rating) = 10 interval [1-10], age interval [0-100], Nkeys(sid)=1000

*SELECT * FROM Sailors WHERE rating = 3 AND age > 50;*

Calculate result size:

NTuples(S) = 1000
RF(rating) = 1/10 = 0.1
RF(age) = (100-50)/(100-0) = 0.5
ResultSize = NTuples(S)*RF(rating)*RF(age)
= 1000*0.1*0.5 = 50 tuples

© University of Melbourne

Homework

Given

Reserves (R): NTuples(R) = 100, Nkeys(sid) =100,
Sailors (S): NTuples(S) =1000, Nkeys(rating) = 10 interval [1-10],
age interval [0-100], Nkeys(sid)=1000

and

SELECT * FROM Sailors as S INNER JOIN Reserves as R
ON S.SID = R.SID WHERE rating = 8 and 20 < age < 30;

Calculate

NTuples(S) = 1000
NTuples(R) = 100
RF(S.SID=R.SID) = ?
RF(rating) = ?
RF(20<age<30) = ?
Result Size = NTuples(S)*NTuples(R)*RF(S.SID=R.SID)*RF(rating)*RF(20<age<30)
= ?

© University of Melbourne